

EOAT Command Center Settings Page Overhaul Plan

Purpose

This document defines a detailed implementation plan for redesigning and expanding the **Settings** page in the EOAT Command Center app.

The current Settings page is functional but too static, too narrow, and no longer matches the maturity of the rest of the app. It currently behaves like a long miscellaneous form instead of a proper configuration control center.

The goal is to turn Settings into a powerful, structured, safe, and extensible control center for:

- Project root and data configuration
- Audit defaults
- Smart conditional defaults
- Scheduled report configuration
- Backup and safety policy
- UI preferences
- External tool configuration
- Diagnostics and advanced tools

This should be built in a clean, maintainable, testable way so future features can plug into Settings without turning `settings.py` into a cursed Python scroll.

1. Current State Summary

1.1 Current Settings Page

The current Settings page appears to include:

- Project Configuration
 - Config file
 - Project root
 - Data mode
 - Master workbook path
- Choose Real Project Folder button
- Git / External Tools

- Git executable
- Browse button
- UI Preferences
- Debug mode
- Theme
- Audit Defaults
- Hardcoded small list of audit default fields
- Connection Defaults
- ATI changeover difficulty
- DoveTail changeover difficulty
- Scheduled Reports
- Placeholder text saying scheduled report editing is not part of this phase
- Safety / Backups
- Placeholder text describing where local files stay
- System Checks / Backups
- Test Git Path
- Save Settings
- Reload Settings
- Run Full System Audit
- Backup Workbook
- Create Light Project Backup
- Open Backups Folder

1.2 Main Problems

The current Settings page has several issues:

1. **It is too static.** The Audit Defaults section is a hardcoded list of fields instead of a flexible default-management system.
2. **It is behind the rest of the app.** Scheduled Reports and Backup Manager already exist as serious app features, but Settings still shows placeholder text.

3. **It lacks typed validation.** Many settings are saved as plain text, even when they should be constrained values, booleans, integers, paths, schedules, or dropdown choices.
 4. **It has no configurable default-rule engine.** The user should be able to add a new default for any known audit field through a "New Default +" workflow.
 5. **It does not support smart conditional defaults.** Examples:
 6. If Part-Present Detection Present? = Yes, set Sensor Type = Reed Switch.
 7. If Part-Present Detection Present? = Yes, set Sensor Brand/Model = SMC.
 8. If Connection Type contains ATI, set Changeover Difficulty = Easy/Low.
 9. If Connection Type contains DoveTail, set Changeover Difficulty = Medium.
 10. **It lacks unsaved-change protection.** Settings should detect dirty state and warn before leaving without saving.
 11. **It lacks settings search.** Once this app grows, users should be able to search for settings like "backup," "schedule," "default," or "theme."
 12. **It mixes simple preferences, project safety controls, and diagnostics into one long vertical form.** That makes the page harder to use and harder to maintain.
-

2. High-Level Goal

Redesign the Settings page into a structured **Settings Control Center**.

The new page should have either a tabbed layout or a two-pane layout with section navigation.

Recommended sections:

1. Project & Data
2. Audit Defaults
3. Smart Rules
4. Scheduled Reports
5. Backups & Safety
6. UI Preferences
7. External Tools
8. Advanced / Diagnostics

Each section should have focused controls, clear descriptions, validation, and safe actions.

3. Major Design Principles

3.1 Preserve Existing Behavior

Do not break existing local config files.

Existing config keys must still load correctly:

- `project_root`
- `debug_mode`
- `theme`
- `git_executable`
- `project_start_date`
- `workdays`
- `skip_weekends`
- `holidays`
- `audit_defaults`
- `connection_defaults`

Existing defaults must continue to work after migration.

3.2 Do Not Commit Real Project Data

The repo must remain safe for GitHub.

Do not commit:

- real workbooks
- real project roots
- local config files
- real reports
- real logs
- real backups
- real machine data
- real press information
- real customer names
- real mold numbers
- internal shared-drive paths
- generated operational outputs

3.3 Settings Should Not Directly Mutate Operational Data

Settings should configure policies and launch safe tools, but it should not directly write into the master workbook except through existing safe functions.

Examples:

- Saving a default rule modifies config only.
- Running “Backup Workbook Now” can call the existing backup tool.
- Installing scheduled tasks can call the existing scheduled report installer.
- Settings should not directly modify audit rows.

3.4 Defaults Must Be Safe

Audit defaults must not overwrite existing real audit data unless the user explicitly requests that.

Default application should follow this behavior by default:

Apply to new audits only.
Apply to empty fields only.
Respect field visibility rules.
Do not populate hidden non-applicable fields.
Do not overwrite saved workbook values.
Do not create stale hidden values.

3.5 UI Should Reflect Truth

Settings must not contain stale placeholder text.

If a feature exists, Settings should either:

- expose its configuration,
- summarize its status,
- link to the deeper page,
- or state clearly why it is unavailable.

No fake claims. No “this is not part of this phase” if the page already exists elsewhere in the app. That is how apps develop trust issues and tiny haunted corners.

4. Proposed Settings Layout

4.1 Top-Level Page Structure

The Settings page should have:

Settings

[Search settings...]

Unsaved changes banner, if needed:

"You have unsaved settings changes."

[Save Settings] [Revert Changes] [Show Changes]

Section navigation:

[Project & Data] [Audit Defaults] [Smart Rules] [Scheduled Reports]

[Backups & Safety] [UI Preferences] [External Tools] [Advanced]

The top-level controls should stay visible or be easy to access:

- Save Settings
- Revert Changes
- Reset Current Section
- Search settings
- Unsaved changes indicator

4.2 Recommended Two-Pane Layout

Preferred layout:

Settings Search [Search...]	Settings
Project & Data	Current section content
Audit Defaults	
Smart Rules	
Scheduled Reports	
Backups & Safety	
UI Preferences	
External Tools	
Advanced	

If a two-pane layout is too much for this phase, use `QTabWidget`.

5. Files and Code Structure

5.1 Current File to Refactor

Current file:

```
app/pages/settings.py
```

This file should not keep growing as one giant class. It should become either a wrapper or a small orchestrator.

5.2 Recommended New Structure

Create:

```
app/pages/settings/  
  __init__.py  
  page.py  
  sections.py  
  models.py  
  widgets.py  
  project_section.py  
  audit_defaults_section.py  
  smart_rules_section.py  
  scheduled_reports_section.py  
  backups_section.py  
  ui_preferences_section.py  
  external_tools_section.py  
  advanced_section.py
```

If converting `app/pages/settings.py` into a package is risky because of import path conflicts, use this safer structure instead:

```
app/pages/settings.py  
app/settings_page/  
  __init__.py  
  sections.py  
  models.py  
  widgets.py  
  project_section.py  
  audit_defaults_section.py  
  smart_rules_section.py  
  scheduled_reports_section.py  
  backups_section.py  
  ui_preferences_section.py  
  external_tools_section.py  
  advanced_section.py
```

Then `app/pages/settings.py` can simply import and expose the main class:

```
from app.settings_page.page import SettingsPage
```

The app currently expects:

```
app.pages.settings:SettingsPage
```

That import path must continue working unless `page_registry.py` is updated safely.

6. Core Config Improvements

6.1 Add Config Schema Version

Add:

```
{  
    "config_schema_version": 2  
}
```

The app should migrate older config files automatically.

6.2 Extend UserConfig

The config model should support these new sections:

```
@dataclass  
class UserConfig:  
    config_schema_version: int = 2  
    project_root: str = str(DEFAULT_PROJECT_ROOT)  
    debug_mode: bool = False  
    theme: str = "light"  
    git_executable: str = str(DEFAULT_GIT_EXECUTABLE)  
    project_start_date: str = ""  
    workdays: list[str] = field(default_factory=lambda: ["Monday", "Tuesday",  
"Wednesday", "Thursday", "Friday"])  
    skip_weekends: bool = True  
    holidays: list[str] = field(default_factory=list)  
  
    audit_defaults: dict[str, str] = field(default_factory=lambda:  
dict(DEFAULT_AUDIT_DEFAULTS))  
    connection_defaults: dict[str, str] = field(default_factory=lambda:
```



```
dict(DEFAULT_CONNECTION_DEFAULTS))

    audit_default_rules: list[dict] = field(default_factory=list)
    smart_default_rules: list[dict] = field(default_factory=list)
    scheduled_reports: dict =
field(default_factory=default_scheduled_reports_config)
    backups: dict = field(default_factory=default_backup_policy_config)
    ui_preferences: dict = field(default_factory=default_ui_preferences_config)
```

6.3 Config Migration Function

Create:

```
core/config_migration.py
```

Example:

```
def migrate_config_dict(data: dict) -> dict:
    migrated = dict(data or {})
    version = int(migrated.get("config_schema_version") or 1)

    if version < 2:
        migrated.setdefault("scheduled_reports",
default_scheduled_reports_config())
        migrated.setdefault("backups", default_backup_policy_config())
        migrated.setdefault("ui_preferences", default_ui_preferences_config())

        # Preserve old audit_defaults.
        old_defaults = migrated.get("audit_defaults", {})
        migrated.setdefault("audit_default_rules",
audit_defaults_dict_to_rules(old_defaults))

        # Preserve old connection_defaults.
        old_connection_defaults = migrated.get("connection_defaults", {})
        migrated.setdefault("smart_default_rules",
connection_defaults_to_smart_rules(old_connection_defaults))

        migrated["config_schema_version"] = 2

    return migrated
```

6.4 Config Example Update

Update:

```
config/config.example.json
```

It should include safe example settings only. No real paths.

Example:

```
{
  "config_schema_version": 2,
  "project_root": "examples/demo_project",
  "debug_mode": false,
  "theme": "light",
  "git_executable": "git",
  "project_start_date": "2026-05-18",
  "workdays": ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday"],
  "skip_weekends": true,
  "holidays": [],
  "scheduled_reports": {
    "daily": {
      "enabled": true,
      "weekdays": ["Monday", "Tuesday", "Wednesday", "Thursday"],
      "time": "19:00",
      "duplicate_policy": "skip_existing",
      "missed_run_policy": "show_as_missed"
    },
    "weekly": {
      "enabled": true,
      "weekday": "Friday",
      "time": "19:00",
      "duplicate_policy": "skip_existing"
    },
    "timezone": "America/New_York",
    "allow_catch_up": true,
    "dry_run_folder_name": "Test_Reports"
  },
  "backups": {
    "auto_backup_before_audit_save": true,
    "auto_backup_before_compatibility_update": true,
    "auto_backup_before_workbook_migration": true,
    "retention": {
      "keep_days": 7,
      "keep_newest_per_workbook": 25,
      "keep_milestones": true
    },
    "cleanup_requires_preview": true,
    "cleanup_blocked_by_validation_errors": true
  }
}
```

```
}  
}
```

7. Settings Schema and Validation

7.1 Add Settings Definition Model

Create:

```
core/settings_schema.py
```

Suggested model:

```
@dataclass(frozen=True)  
class SettingDefinition:  
    key: str  
    label: str  
    section: str  
    value_type: str  
    default: object  
    description: str = ""  
    choices: tuple[str, ...] = ()  
    requires_restart: bool = False  
    advanced: bool = False
```

Possible `value_type` values:

```
string  
boolean  
integer  
enum  
path  
time  
weekday_list  
json  
field_default  
rule_list
```

7.2 Add Validation

Create:

```
core/settings_validation.py
```

Validation should check:

- theme is one of known themes
- times are valid `HH:MM`
- weekdays are valid weekday names
- project root is a string path
- backup retention numbers are positive integers
- scheduled report duplicate policy is known
- yes/no defaults are valid
- numeric audit defaults are numeric when required
- dropdown audit defaults match allowed values when strict
- rule conditions reference valid known fields when possible

7.3 User-Friendly Validation Results

Create a result object:

```
@dataclass
class SettingsValidationIssue:
    severity: str # error, warning, info
    section: str
    field: str
    message: str
    suggestion: str = ""
```

Settings should refuse save if there are errors.

Warnings can be saved but should be shown.

8. Project & Data Section

8.1 Purpose

This section controls what project the app is operating on.

8.2 Display

Show compact status cards:

```
Data Mode: Real Project / Demo / Missing / Invalid
Project Root: <path>
Master Workbook: Found / Missing
Config File: <path>
```

8.3 Controls

Buttons:

```
[Choose Real Project Folder]
[Switch to Demo Project]
[Open Project Root]
[Open Master Workbook]
[Validate Project Foundation]
[Copy Project Paths]
```

8.4 Expected Behavior

- Choosing a project folder updates the text field but does not save until Save Settings is clicked.
- Validate Project Foundation can run before saving to check if the selected root is valid.
- Switching project roots should trigger app-wide project root changed events after save.
- Project root should not silently fall back to demo if the real project is invalid. It should show Missing or Invalid clearly.

9. Audit Defaults Section

9.1 Purpose

Replace the current fixed list of Audit Defaults with a flexible manager.

The user should be able to define defaults for any known audit field, not just the small hardcoded list.

9.2 Current Problem

Current Settings uses a hardcoded list:

Auditor
Plant/Area
Cleanroom/Non-Cleanroom
Status
Priority
Follow-Up Needed
Quick Disconnects Present?
Pneumatic Quick Disconnect Type
Vacuum Generator Type
EOAT Interchangeable Circuits
Robot Interchangeable Circuits
Photos Taken?

But the default system already includes more fields. The UI should expose defaults more flexibly.

9.3 UI Layout

Audit Defaults

These defaults apply to new audits unless configured otherwise. Defaults should not overwrite saved audit data unless explicitly requested.

[+ New Default] [Edit Selected] [Disable Selected] [Delete Selected]
[Preview Applied Defaults] [Import Defaults] [Export Defaults] [Reset System Defaults]

Table:

Enabled	Field	Default Value	Scope	Overwrite Policy	Conditions	Source	Notes
---------	-------	---------------	-------	------------------	------------	--------	-------

9.4 New Default Dialog

When the user clicks **New Default +**, show:

New Audit Default

Field:

[searchable dropdown of known audit fields]

Default value:

[dynamic editor based on field type]

Scope:

[New audits only]

[Existing empty fields only]
[Manual apply only]

Overwrite policy:
[Empty only]
[Ask before replacing]
[Never overwrite]

Conditions:
[+ Add Condition]

Enabled:
[x]

Notes:
[optional text]

[Save Default] [Cancel]

9.5 Field Selection

The field dropdown should include known audit fields from the audit schema/form sections.

Fields include, but are not limited to:

Audit ID
Audit Date
Auditor
Plant/Area
Press/Machine #
Status
Priority
Follow-Up Needed
Robot Type
Robot Model/Controller
Tool #
Part Family
Part Name/Description
Cleanroom/Non-Cleanroom
EOAT Type
EOAT Moves
Connection Type
Number of Parts Picked
of Grippers
Gripper Type
Gripper Model

Gripper Size
Number of Vacuum Cups
Cup Type/Material
Cup Diameter/Size
Vacuum Generator Type
Estimated EOAT Weight
EOAT Vacuum Circuits
EOAT Pressure Circuits
EOAT Interchangeable Circuits
Robot Vacuum Circuits
Robot Pressure Circuits
Robot Interchangeable Circuits
Sensors Present?
Sensor Type
Sensor Brand/Model
Vacuum Confirmation Present?
Part-Present Detection Present?
Electrical/Wiring Present?
Quick Disconnects Present?
Pneumatic Quick Disconnect Type
Electrical Quick Disconnect Type
Tubing Condition
Tubing Routing Notes
Cable Management Condition
Mounting Hardware Condition
EOAT Alignment Condition
Fastener/Locking Hardware Present?
Known Issues
Drop/Mis-Pick History
Maintenance Frequency
Cycle Time Concern?
Scrap/Quality Concern?
Changeover Difficulty
Spare Parts Identified?
Drawing/CAD Available?
BOM Available?
Process Binder Complete?
Photos Taken?
Photo Folder/Link
Pilot Candidate?
Notes

The final field list should come from the app's existing audit schema instead of a duplicated hardcoded list wherever possible.

9.6 Default Rule Data Model

Create:

```
core/audit/default_rules.py
```

Suggested data model:

```
@dataclass
class RuleCondition:
    field: str
    operator: str
    value: str = ""

@dataclass
class AuditDefaultRule:
    id: str
    enabled: bool
    field: str
    value: str
    scope: str = "new_audit"
    overwrite_policy: str = "empty_only"
    conditions: list[RuleCondition] = field(default_factory=list)
    source: str = "user"
    note: str = ""
```

Allowed `scope` values:

```
new_audit
existing_empty_fields
manual_only
```

Allowed `overwrite_policy` values:

```
empty_only
ask
never
```

Allowed condition operators:

```
equals
not_equals
contains
not_contains
is_empty
is_not_empty
in_list
```

9.7 Default Rule Application

The default engine should provide:

```
def applicable_default_rules(config, form_values, visible_fields, mode) ->
list[AuditDefaultRule]:
    ...

def apply_default_rules(config, form_values, visible_fields, mode, *,
overwrite=False) -> dict[str, str]:
    ...
```

Default behavior:

- Use only enabled rules.
- Apply only to visible/applicable fields.
- Apply only in allowed scope.
- Apply only when conditions match.
- Apply only to empty values unless explicit overwrite is allowed.
- Do not populate fields hidden by EOAT visibility logic.
- Do not overwrite workbook-loaded values by default.

9.8 Preview Applied Defaults

Add a preview button that shows:

Defaults that would apply to a new audit:

Field	Current	Default	Reason
Auditor	blank	Kato Gray	New audit default
Plant/Area	blank	Plant 4	New audit default
Sensor Type	blank	Reed Switch	Part-Present Detection
Present? = Yes			

This preview should not save anything.

10. Smart Rules Section

10.1 Purpose

Smart Rules are conditional auto-fill rules.

They are similar to defaults, but explicitly triggered by other field values.

10.2 UI Layout

Smart Rules

[+ New Rule] [Edit Selected] [Disable Selected] [Delete Selected] [Test Rule]

Table:

Enabled | Trigger | Action | Applies | Overwrite Policy | Notes

10.3 Example Rules

Include these default smart rules after migration or as system suggestions:

Part-present sensor defaults

If Part-Present Detection Present? equals Yes:
Set Sensor Type to Reed Switch
Overwrite policy: Empty only

If Part-Present Detection Present? equals Yes:
Set Sensor Brand/Model to SMC
Overwrite policy: Empty only

Connection changeover defaults

If Connection Type contains ATI:
Set Changeover Difficulty to Low or Easy
Overwrite policy: Empty only

If Connection Type contains DoveTail:
Set Changeover Difficulty to Medium
Overwrite policy: Empty only

10.4 Rule Builder Dialog

New Smart Rule

When:

Field: [dropdown]

Operator: [equals / contains / is empty / is not empty / etc.]

Value: [input]

Then:

Set Field: [dropdown]

To Value: [input/dropdown]

Apply:

[New audits]

[Loaded existing audits with empty field]

[Manual only]

Overwrite:

[Empty only]

[Ask first]

[Never]

Enabled:

[x]

Notes:

[optional]

10.5 Rule Safety

Smart rules must:

- Not overwrite user-entered values unless configured.
- Not apply to hidden fields.
- Not create stale hidden field values.
- Not conflict silently.

If two rules set the same field to different values, show a warning:

Two enabled rules can set "Changeover Difficulty" under overlapping conditions. Review rule priority or disable one.

10.6 Rule Priority

Optional but useful:

Priority	Rule
1	Part-present detection sets sensor type
2	Connection type sets changeover difficulty

If priorities are not implemented, conflicts should still be detected.

11. Scheduled Reports Section

11.1 Purpose

Settings should configure scheduled reports instead of showing stale placeholder text.

The app already has a Scheduled Reports page for diagnostics and manual report actions. Keep that page.

Settings should provide:

- schedule configuration
- summary status
- quick actions
- link to detailed Scheduled Reports page

11.2 New Config

Add:

```
"scheduled_reports": {
  "daily": {
    "enabled": true,
    "weekdays": ["Monday", "Tuesday", "Wednesday", "Thursday"],
    "time": "19:00",
    "duplicate_policy": "skip_existing",
    "missed_run_policy": "show_as_missed"
  },
  "weekly": {
```

```

    "enabled": true,
    "weekday": "Friday",
    "time": "19:00",
    "duplicate_policy": "skip_existing"
  },
  "timezone": "America/New_York",
  "allow_catch_up": true,
  "dry_run_folder_name": "Test_Reports"
}

```

11.3 UI Layout

Scheduled Reports

Daily Summary

☒ Enabled

Weekdays: [Monday] [Tuesday] [Wednesday] [Thursday] [Friday]

Time: [19:00]

Duplicate policy: [Skip existing report]

Missed run policy: [Show as missed]

Weekly Summary

☒ Enabled

Weekday: [Friday]

Time: [19:00]

Duplicate policy: [Skip existing report]

Timezone:

[America/New_York]

Actions:

[Install/Repair Scheduled Tasks]

[Uninstall Scheduled Tasks]

[Run Daily Dry Run]

[Run Weekly Dry Run]

[Generate Daily Now]

[Generate Weekly Now]

[Open Scheduled Tool Log]

[Open Scheduled Reports Page]

11.4 Status Cards

Show:

```
Daily Task Installed: Yes/No/Unknown
Daily Last Run: timestamp or none
Daily Next Run: timestamp or none
Weekly Task Installed: Yes/No/Unknown
Weekly Last Run: timestamp or none
Weekly Next Run: timestamp or none
Scheduled Log: found/missing
```

11.5 Scheduled Reports Core Refactor

Currently schedule behavior is mostly constant-driven. Refactor so the core scheduled report logic reads schedule settings from config.

Add:

```
def resolve_scheduled_report_settings(config_or_project_root) ->
  ScheduledReportSettings:
    ...
```

The existing Scheduled Reports page should use the same resolved settings.

Do not duplicate schedule logic between Settings and Scheduled Reports page.

11.6 Important Behavior

- Saving schedule settings does not automatically install Windows Scheduled Tasks unless user clicks Install/Repair or confirms.
- If schedule settings changed and installed task is outdated, show warning:

```
Schedule settings changed. Windows Scheduled Tasks may need repair.
[Install/Repair Tasks]
```

- Dry runs should not overwrite real reports.
- Existing same-day reports should still be detected and not overwritten unless policy changes later.

12. Backups & Safety Section

12.1 Purpose

Settings should expose backup and safety policies.

The app already has a Backup Manager page. Keep that page as the detailed backup inventory and cleanup tool.

Settings should provide:

- backup policy config
- backup safety status
- quick actions
- link to Backup Manager

12.2 New Config

Add:

```
"backups": {
  "auto_backup_before_audit_save": true,
  "auto_backup_before_compatibility_update": true,
  "auto_backup_before_workbook_migration": true,
  "auto_backup_before_bulk_repair": true,
  "retention": {
    "keep_days": 7,
    "keep_newest_per_workbook": 25,
    "keep_milestones": true
  },
  "cleanup_requires_preview": true,
  "cleanup_blocked_by_validation_errors": true
}
```

12.3 UI Layout

Backups & Safety

Automatic Backups

- ☒ Backup before audit save
- ☒ Backup before compatibility update
- ☒ Backup before workbook migration/repair
- ☒ Backup before bulk workbook repair

Retention Policy

- Keep backups from last [7] days
- Keep newest [25] per workbook
- ☒ Keep milestone backups

Cleanup Safety

- ☒ Cleanup requires preview

[x] Block cleanup when validation has blockers

Actions:

[Create Workbook Backup Now]
[Create Light Project Backup Now]
[Create Emergency Backup ZIP]
[Preview Backup Cleanup]
[Open Backup Manager]
[Open Backups Folder]
[Open Logs Folder]

12.4 Safety Rules

Backup cleanup must always be safe.

Rules:

- Never delete without explicit confirmation.
- Never delete without preview if `cleanup_requires_preview` is true.
- Refuse cleanup if validation blockers exist and `cleanup_blocked_by_validation_errors` is true.
- Show exactly what would be deleted.
- Do not hide warnings.

12.5 Emergency Backup ZIP

Add an optional tool action:

Create Emergency Backup ZIP

This should create a timestamped lightweight backup package of important project outputs.

It should include safe local project files but must remain outside the repo.

Do not commit emergency backups.

13. UI Preferences Section

13.1 Purpose

Move simple visual and app interaction settings into a focused UI section.

13.2 Controls

```
Theme:  
[Light / Dark]  
  
Debug mode:  
[x]  
  
Page density:  
[Comfortable / Compact]  
  
Output panel default:  
[Expanded / Collapsed / Remember last state]  
  
Show advanced controls:  
[x]  
  
Confirm before closing pages with unsaved edits:  
[x]  
  
Show tooltips/help text:  
[x]
```

13.3 Optional Later Features

These are not required immediately but should be considered:

- remember window size
- remember last opened page
- remember sidebar expanded/collapsed groups
- command palette behavior
- table row density
- high contrast mode

14. External Tools Section

14.1 Purpose

Configure tool paths and verify external dependencies.

14.2 Controls

```
Git executable:
[path/input]
[Browse] [Test Git Path]

PowerShell executable:
[path/input or auto]
[Test PowerShell]

Python executable:
[path/input or current interpreter]
[Test Python]

Actions:
[Run Tool Environment Check]
[Copy Tool Diagnostics]
```

14.3 Git Test

Current Git test behavior should remain, but the output should be more structured:

```
Git executable: git
Executable found: Yes
Version: git version x.x.x
Project root is Git repo: Yes/No
Warning: ...
```

15. Advanced / Diagnostics Section

15.1 Purpose

Keep dangerous or developer-focused actions out of the normal sections.

15.2 Controls

```
[Run Full System Audit]
[Clear Dashboard Cache]
[Open Cache Folder]
[Open Logs Folder]
```

[Export Diagnostics Bundle]
[Open Local Config JSON]
[Copy Settings Summary]
[Reset Local Config]

15.3 Reset Local Config

This must require confirmation.

Confirmation text:

This will reset local app settings such as project root, theme, defaults, scheduled report settings, and backup policies. It will not delete project files or workbooks.

Are you sure?

Do not delete real project data.

16. Search Settings

16.1 Purpose

Add a search bar at the top of Settings.

16.2 Behavior

Typing filters visible sections/settings.

Examples:

Search:

Show:

- Backups & Safety section
- backup before audit save
- backup retention
- open backup folder
- backup cleanup
- Backup Manager link

Search: default

Show:

- Audit Defaults
- Smart Rules
- New Default
- Reset defaults
- Preview applied defaults

Search: schedule

Show:

- Scheduled Reports
- daily summary settings
- weekly summary settings
- install/repair tasks
- scheduled log

16.3 Highlighting

Optional:

- highlight matching labels
 - show section match counts
-

17. Dirty-State Tracking

17.1 Purpose

Settings should detect unsaved edits.

17.2 Behavior

When a field changes:

- Set page dirty state to true.
- Show banner:

You have unsaved settings changes.
[Save Settings] [Revert Changes] [Show Changes]

- Enable Save button.
- Enable Revert button.

When Save succeeds:

- Clear dirty state.
- Update baseline.
- Emit settings saved event.
- If project root changed, emit project root changed event.

When Revert is clicked:

- Restore last loaded config.
- Clear dirty state.

17.3 Navigation Guard

If the user tries to leave Settings with unsaved changes:

Show:

You have unsaved settings changes.

[Save and Leave]
[Leave Without Saving]
[Cancel]

17.4 Change Summary

Show changed values:

Changed Settings:

- Theme: Light → Dark
- Daily Summary Time: 19:00 → 18:30
- Added audit default: Cylinder Type = Linear
- Backup retention keep days: 7 → 14

18. Default Field Type Handling

18.1 Known Field Types

Create a central field metadata provider if one does not already exist.

Suggested:

```
core/audit/field_metadata.py
```

Example:

```
@dataclass(frozen=True)
class AuditFieldMetadata:
    name: str
    label: str
    section: str
    value_type: str
    choices: tuple[str, ...] = ()
    default: str = ""
    required: bool = False
    notes_count_excluded: bool = False
```

18.2 Editor Selection

Field type to editor:

```
yes_no -> combo ["", "Yes", "No", "Unknown / Not Checked"]
integer -> spinbox or line edit with integer validator
dropdown -> combo
free_text -> line edit
long_text -> text edit
path -> path picker
```

18.3 Examples

Fields:

```
# of Cylinders -> integer
Cylinder Type -> dropdown ["Linear", "Rotary"]
Photos Taken? -> yes/no
```

Priority -> dropdown ["Low", "Medium", "High"]
Status -> dropdown ["In Progress", "Completed", etc.]
Notes -> long text

19. Add Cylinder Defaults Support

The app is adding Cylinder fields to EOAT Type and Tooling.

Settings default manager should support:

of Cylinders
Cylinder Type

Default suggestion:

Cylinder Type = Linear

Important:

- Cylinder section should never disappear.
- If both cylinder fields are empty, they should not count against audit coach completion.
- If either cylinder field is filled, the section participates as normal.
- Settings should allow defaults for these fields, but defaulting # of Cylinders to 0 may create ambiguity. Prefer leaving count blank by default unless user explicitly sets it.

20. Audit Coach Exclusion Settings

20.1 Purpose

Some fields should not count against completion percentage.

Examples already discussed:

Tubing Routing Notes
Notes
Cylinder fields when both are empty

Source Audit ID
Compatibility Source

20.2 Add Settings UI

Create a subsection under Smart Rules or Audit Defaults:

Audit Completion Rules

Fields excluded from completion:

- ☒ Tubing Routing Notes
- ☒ Notes
- ☒ Source Audit ID
- ☒ Compatibility Source

Special rules:

- ☒ Ignore Cylinder section when # of Cylinders and Cylinder Type are both empty
- ☒ Ignore hidden non-applicable fields
- ☒ Ignore robot-side fields that are stored in Robot_Info.xlsx when not applicable

20.3 Important

Completion settings must remain truthful.

Do not exclude important fields just to inflate the score. The purpose is to avoid penalizing intentionally unused or non-applicable fields.

21. Connection Defaults Replacement

21.1 Current Behavior

The app currently exposes connection defaults as:

ATI changeover difficulty
DoveTail changeover difficulty

This is too narrow.

21.2 New Behavior

Convert connection defaults into Smart Rules.

Examples:

```
If Connection Type contains ATI:  
Set Changeover Difficulty = Low
```

```
If Connection Type contains DoveTail:  
Set Changeover Difficulty = Medium
```

21.3 Migration

During config migration:

- Preserve existing `connection_defaults`.
- Convert each entry into a smart rule.
- Keep old `connection_defaults` for backward compatibility for now.
- Prefer smart rules in the UI.

22. Scheduled Reports Integration Details

22.1 Existing Scheduled Reports Page

Keep the current Scheduled Reports page.

Settings should link to it:

```
[Open Scheduled Reports Page]
```

The detailed page should remain the place for:

- calendar preview
- missed runs
- catch-up selected
- preflight diagnostics
- status cards
- latest report opening

22.2 Settings Should Not Duplicate Everything

Settings should expose configuration and basic status.

Detailed operations remain on Scheduled Reports page.

22.3 Config-Driven Schedule

Scheduled Reports core should no longer depend only on constants for schedule decisions.

Add a config resolver:

```
@dataclass(frozen=True)
class ScheduledReportsConfig:
    daily_enabled: bool
    daily_weekdays: tuple[int, ...]
    daily_time: time
    weekly_enabled: bool
    weekly_weekday: int
    weekly_time: time
    timezone: str
    duplicate_policy: str
    missed_run_policy: str
    dry_run_folder_name: str
```

23. Backup Manager Integration Details

23.1 Existing Backup Manager Page

Keep Backup Manager as the deep page.

Settings should link to it:

```
[Open Backup Manager]
```

23.2 Settings Should Provide Policies

Settings controls policies.

Backup Manager performs inventory, preview, and cleanup.

23.3 Backup Policy Resolver

Create:

```
def resolve_backup_policy(config) -> BackupPolicy:  
    ...
```

Existing backup tools should read this policy where appropriate.

24. Save / Reload Behavior

24.1 Current Behavior

Current Settings save updates config fields and calls `save_config`.

New behavior should:

1. Validate all settings.
2. Show errors if invalid.
3. Save only if valid.
4. Persist config.
5. Update internal baseline.
6. Emit relevant events.
7. Show clear result summary.

24.2 Save Result Example

Settings saved.

Changed:

- Added audit default: Cylinder Type = Linear
- Daily summary time changed to 7:00 PM
- Backup before compatibility update enabled

Config file:

config/local_config.json

Warnings:

- Scheduled report settings changed. Install/repair tasks to apply Windows Task Scheduler changes.

25. Import / Export Defaults

25.1 Purpose

Allow sharing/reuse of default profiles without editing JSON manually.

25.2 Export

Button:

[Export Defaults]

Exports only default rules and smart rules, not project root or local paths.

Example file:

```
{
  "type": "eoat_command_center_defaults",
  "version": 1,
  "audit_default_rules": [],
  "smart_default_rules": []
}
```

25.3 Import

Button:

[Import Defaults]

Import should:

- validate file type
- preview changes
- allow merge or replace
- avoid duplicate rule IDs
- not import project root paths

Options:

```
[Merge with existing]
[Replace existing defaults]
[Cancel]
```

26. Profiles / Presets

26.1 Optional But Recommended

Add named settings profiles later.

Example profiles:

```
Plant 4 Whiteroom Default
Plant 4 Non-Cleanroom Default
Demo Profile
Temporary Audit Session
```

A profile can include:

- audit default rules
- smart rules
- UI preference subset
- maybe selected project root, but be careful with local path safety

26.2 Phase Later

This does not need to be in the first implementation, but the settings schema should not make it impossible.

27. Tests Required

Add tests for config, rules, and UI behavior where possible.

27.1 Suggested Test Files

```
tests/test_settings_config_migration.py
tests/test_settings_validation.py
tests/test_audit_default_rules.py
tests/test_smart_default_rules.py
```

```
tests/test_scheduled_report_settings.py
tests/test_backup_policy_settings.py
tests/test_settings_dirty_state.py
```

27.2 Specific Tests

Config Migration

```
test_old_config_without_schema_version_loads
test_migration_preserves_project_root
test_migration_preserves_audit_defaults
test_migration_preserves_connection_defaults
test_migration_adds_scheduled_reports_defaults
test_migration_adds_backup_policy_defaults
test_migration_is_idempotent
```

Audit Default Rules

```
test_default_rule_applies_to_new_audit_empty_field
test_default_rule_does_not_overwrite_existing_value
test_disabled_default_rule_does_not_apply
test_default_rule_respects_visible_fields
test_default_rule_does_not_apply_to_hidden_field
test_default_rule_condition_equals_matches
test_default_rule_condition_equals_does_not_match
test_default_rule_condition_contains_matches
test_default_rule_condition_is_empty_matches
test_default_rule_preview_returns_expected_rows
```

Smart Rules

```
test_part_present_detection_sets_reed_switch_when_empty
test_part_present_detection_sets_smc_when_empty
test_part_present_detection_does_not_overwrite_custom_sensor
test_connection_type_ati_sets_changeover_difficulty
test_connection_type_dovetail_sets_changeover_difficulty
test_conflicting_smart_rules_generate_warning
```

Scheduled Reports

```
test_default_scheduled_report_config_matches_current_schedule
test_custom_daily_time_is_loaded
```

```
test_custom_daily_weekdays_are_loaded
test_custom_weekly_day_is_loaded
test_scheduled_report_settings_validation_rejects_bad_time
test_scheduled_report_settings_validation_rejects_bad_weekday
```

Backup Policy

```
test_default_backup_policy_loads
test_backup_policy_rejects_negative_retention_days
test_backup_policy_rejects_zero_keep_newest
test_cleanup_requires_preview_defaults_true
test_cleanup_blocked_by_validation_defaults_true
```

Dirty State

```
test_settings_dirty_after_field_change
test_settings_clean_after_save
test_settings_revert_restores_baseline
test_settings_change_summary_lists_changed_values
```

28. Acceptance Criteria

The implementation is complete when:

1. Settings page no longer shows stale placeholder text for Scheduled Reports or Safety / Backups.
2. Settings is organized into clear sections:
3. Project & Data
4. Audit Defaults
5. Smart Rules
6. Scheduled Reports
7. Backups & Safety
8. UI Preferences
9. External Tools
10. Advanced / Diagnostics
11. A **New Default +** workflow exists.
12. The user can create a default for any known audit field.

- 13. Default rules support:
- 14. enabled/disabled
- 15. field
- 16. value
- 17. scope
- 18. overwrite policy
- 19. optional conditions
- 20. notes
- 21. Defaults apply safely:
- 22. new audits only by default
- 23. empty fields only by default
- 24. visible/applicable fields only
- 25. no hidden stale data
- 26. no overwriting saved workbook values unless explicitly requested
- 27. Smart Rules exist for conditional defaults.
- 28. Scheduled report schedule settings are configurable from Settings.
- 29. Backup and safety policies are configurable from Settings.
- 30. Settings has dirty-state tracking.
- 31. Settings has Save, Revert, and Reset Section behavior.
- 32. Settings has search/filtering.
- 33. Existing local config files still load.
- 34. Existing audit defaults still work.
- 35. Existing connection defaults are preserved or migrated into smart rules.
- 36. Scheduled Reports page still works.
- 37. Backup Manager page still works.
- 38. No real project data is committed.

39. `python -m pytest` passes.

40. The UI remains responsive. Expensive checks run in background tasks.

29. Recommended Implementation Order

Phase 1: UI Restructure

- Refactor Settings into sections.
- Keep current behavior.
- Remove stale placeholder text.
- Add section navigation or tabs.
- Add search box stub.
- Add Save/Revert structure.

Phase 2: Config Schema and Migration

- Add config schema version.
- Add migration function.
- Add `scheduled_reports` and `backups` config.
- Update config example.
- Add migration tests.

Phase 3: Audit Default Manager

- Add table-based default manager.
- Add New Default dialog.
- Add rule model.
- Add save/load.
- Add preview behavior.
- Add tests.

Phase 4: Smart Rules

- Add conditional smart rule model.
- Add rule builder dialog.
- Add default smart rules for part-present and connection type.
- Add conflict warnings.
- Add tests.

Phase 5: Scheduled Reports Settings

- Make schedule configurable.
- Update core scheduled report logic to read config.

- Add Settings controls.
- Add status/actions.
- Add tests.

Phase 6: Backups & Safety Settings

- Add backup policy config.
- Add Settings controls.
- Wire to existing backup manager/tooling where appropriate.
- Add tests.

Phase 7: Dirty-State and Search Polish

- Finish dirty-state tracking.
- Add navigation guard.
- Add change summary.
- Finish search/filter behavior.

Phase 8: Final QA

Run:

```
python -m pytest
python scripts/repo_safety_audit.py --root .
python run_dashboard.py
```

Manual checks:

- Open Settings.
 - Search for “backup.”
 - Search for “default.”
 - Add new default.
 - Revert unsaved changes.
 - Save settings.
 - Reload settings.
 - Change scheduled report time.
 - Confirm warning to install/repair tasks.
 - Create backup from Settings.
 - Open Backup Manager from Settings.
 - Open Scheduled Reports from Settings.
 - Confirm no real paths or data are staged.
-

30. Short Prompt to Start Codex

Use this short prompt with this document:

```
Implement the Settings Page Overhaul described in the attached document. Start with Phase 1 and Phase 2 only: restructure the Settings UI into sections/tabs, remove stale placeholder text, add config schema versioning/migration, add scheduled_reports and backups config defaults, preserve all existing config behavior, and add tests. Do not implement the full Default Manager yet unless Phase 1 and 2 are complete and passing. Keep app.pages.settings:SettingsPage working. Do not commit real project data. Run python -m pytest when done.
```

31. Important Notes for Codex

Do not try to build every phase in one giant commit.

Recommended commits:

1. Refactor Settings page structure
2. Add config schema version and migration
3. Add Settings validation foundation
4. Add Audit Default Manager
5. Add Smart Rules
6. Add Scheduled Reports settings integration
7. Add Backup policy settings integration
8. Add Settings search and dirty-state polish

Each commit should have tests.

Do not break the dashboard page registry.

Do not break existing app startup.

Do not make Settings slower.

Do not block UI while checking schedules, backups, or system audits.

Do not overwrite local config without preserving unknown keys if possible.

Do not stage local config files.

Do not put real shared-drive paths into committed examples.

Do not create fake status claims. If task status cannot be determined, show Unknown with a reason.

The Settings page should become the app's configuration command center, not a vertical junk drawer with a Save button stapled to the bottom.